



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): René Adrián Dávila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: 7

No de Práctica(s): 2

Integrante(s): 323078401

323115296

323082392

323139441

323253916

*No. de lista o
brigada:*

4

Semestre: 2027 - I

Fecha de entrega: 08/09/2026

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción	2
2. Marco Teórico	2
3. Desarrollo	3
3.1. Ejercicio 1	3
3.2. Análisis de la respuesta 1	4
3.3. Pruebas de la respuesta 1	4
3.4. Observaciones	4
3.5. Ejercicio 2: Número mayor	5
3.6. Prueba realizada	5
3.7. Ejercicio 3: Generación de una tabla	5
3.8. Prueba realizada	6
4. Resultados	7
5. Conclusión	8
6. Referencias	8

1. Introducción

- **Planteamiento del problema.**

Se requiere plantear un problema a los LLM para deducir la lógica de un programa sin introducir más que una descripción del programa. Por otro lado, se requiere crear dos programas en Java: uno para la captura y la determinación del valor más grande mediante el uso de variables acumuladoras, y el otro para la generación de datos en un formato tabular mediante iteraciones y secuencias de escape.

- **Motivación.** Es importante realizar estos ejercicios porque debemos saber que tan confiable son los LLM al dar respuesta a algún problema planteado, viendo como da diferentes resultados a cada persona para resolver estos problemas, además de ejercitar conceptos adquiridos de estructura de datos y como los implementamos en otros paradigmas.

- **Objetivos.**

- Comparar los resultados obtenidos de diferentes LLM para el primer ejercicio.
- Identificar el número mayor y menor entre 10 valores.
- Mostrar una tabla de resultados a partir de un número N.
- Analizar la solución propuesta por el profesor y compararla con la obtenida mediante un LLM.

2. Marco Teórico

Programación Estructurada y Control de Flujo La programación estructurada es un paradigma que permite organizar un programa mediante instrucciones que siguen un orden determinado. Para esto se utilizan secuencias, decisiones y repeticiones, lo que facilita seguir la ejecución del código. El control de flujo se encarga de determinar el orden en que se ejecutan las instrucciones, permitiendo repetir bloques o tomar diferentes caminos dependiendo de una condición [1].

Estructuras Condicionales Las estructuras condicionales permiten ejecutar diferentes instrucciones dependiendo de si una condición se cumple o no. En Java se pueden utilizar estructuras como `if` e `if-else`, que permiten tomar una decisión a partir de una condición. También existe `switch`, que permite seleccionar una opción entre varios casos posibles [2].

Estructuras de Repetición (Ciclos) Los ciclos permiten repetir un conjunto de instrucciones mientras se cumpla una condición o durante un número determinado de veces. Para utilizarlos correctamente es necesario establecer una condición que permita terminar el ciclo [4].

Ciclo `while`: primero evalúa la condición y, si esta es verdadera, ejecuta las instrucciones que se encuentran dentro del ciclo. Puede utilizarse junto con un contador para controlar las repeticiones [4].

Ciclo `-while`: ejecuta primero las instrucciones y después comprueba la condición. Por esta razón, el bloque se ejecuta al menos una vez [4].

Ciclo : permite establecer en una misma estructura la inicialización de una variable, la condición que controla el ciclo y el incremento. Es útil cuando se conoce el número de repeticiones que se necesitan [2].

Entrada y Salida Estándar en Java La entrada estándar `System.in` permite recibir información introducida por el usuario desde la consola. Para facilitar la lectura de estos datos se puede utilizar la clase `Scanner`, que permite obtener diferentes tipos de datos mediante métodos como `nextInt()` para valores enteros [3]. Para mostrar información en la consola se utiliza `System.out`. También se pueden utilizar caracteres especiales como `\t`, que representa una tabulación y permite separar los datos al momento de imprimirlos [3].

Prompts en Modelos de Lenguaje y Análisis de Código Los prompts son las instrucciones que se proporcionan a un modelo de lenguaje para obtener una respuesta. En programación, la forma en que se plantea un problema puede cambiar la solución generada. Incluir datos, condiciones y restricciones específicas ayuda a reducir interpretaciones diferentes del problema [1]. En el caso del análisis realizado en esta práctica, se pudo observar que cuando el planteamiento no especificaba algún dato, la LLM podía asumir una condición por su cuenta. Por ello, los detalles incluidos en el prompt pueden influir directamente en la solución propuesta [3].

3. Desarrollo

3.1. Ejercicio 1

En este ejercicio se utilizaron tres LLM para analizar cómo cada una interpretaba y resolvía el mismo problema planteado por el profesor. El problema considera diez alumnos y requiere determinar la cantidad de aprobados y reprobados para mostrar un mensaje dependiendo del resultado obtenido.

Para el análisis se tomaron cuatro casos principales. Cuando los diez alumnos aprueban, se muestra un mensaje de felicitación acompañado de una bonificación al instructor. Cuando nueve alumnos aprueban y uno reprueba, se mantiene el mensaje de felicitación y la bonificación. Si ocho alumnos aprueban y dos reprueban, se muestra el mensaje de felicitación, pero no se otorga la bonificación. Finalmente, cuando todos los alumnos reprueban, se muestra un mensaje diferente.

3.2. Análisis de la respuesta 1

La primera LLM recibió un planteamiento descrito de manera informal y a partir de los casos de prueba tuvo que deducir la lógica del problema. La respuesta interpretó que la condición para otorgar la bonificación era que existiera como máximo un alumno reprobado.

La solución propuesta utilizó una entrada mediante **Scanner**, un ciclo **for** para procesar a los diez alumnos y un contador denominado **reprobados**. Al finalizar el ciclo, se evaluó si la cantidad de reprobados era menor o igual a uno para determinar si debía mostrarse la bonificación.

Una de las principales decisiones de la LLM fue asumir que una calificación mayor o igual a 6 representaba una aprobación. Esta condición no se encontraba especificada directamente en el planteamiento recibido, por lo que corresponde a una suposición de la respuesta.

3.3. Pruebas de la respuesta 1

Se consideraron los siguientes casos:

- Diez alumnos aprobados: se muestra la felicitación y la bonificación.
- Nueve alumnos aprobados y uno reprobado: se muestra la felicitación y la bonificación.
- Ocho alumnos aprobados y dos reprobados: se muestra la felicitación, pero no la bonificación.
- Diez alumnos reprobados: se muestra un mensaje diferente.

La respuesta reproduce los casos descritos mediante la condición relacionada con la cantidad de alumnos reprobados.

3.4. Observaciones

La respuesta logró deducir una condición general a partir de los casos proporcionados. Sin embargo, la falta de especificaciones en el prompt provocó que la LLM tuviera que asumir el criterio de aprobación.

También agregó elementos que no eran indispensables para resolver el problema, como una tabla de escenarios, un diagrama de flujo y una explicación sobre el uso de arreglos. Estos elementos pueden ayudar a comprender la solución, pero no formaban parte de los requerimientos iniciales.

En términos de implementación, la respuesta procesó los datos de manera secuencial y utilizó un contador para determinar la cantidad de alumnos reprobados. Esto permite tomar la decisión sobre la bonificación una vez terminada la captura de los diez datos.

3.5. Ejercicio 2: Número mayor

Para resolver el ejercicio se utilizó la clase **Scanner**, con la finalidad de recibir datos ingresados por el usuario mediante la entrada estándar. Posteriormente, se creó la clase principal **Mayor** y se inicializó el método **main**, dentro del cual se estableció el objeto **Scanner** para capturar los datos.

Se declararon tres variables de tipo entero: **cont**, **num** y **mayor**. La variable **cont** funciona como contador para controlar la cantidad de números ingresados, **num** almacena temporalmente cada número proporcionado por el usuario y **mayor** conserva el valor más grande encontrado durante el proceso.

Para solicitar los diez números se utilizó un ciclo **while**, el cual permite repetir el proceso mientras el contador sea menor que diez. En cada iteración se solicita un número entero y se almacena en la variable correspondiente.

Después de recibir cada número, se verifica si es negativo. En caso de serlo, se cambia su signo mediante una multiplicación por -1 , obteniendo así su valor absoluto para realizar la comparación únicamente con valores positivos.

Posteriormente, se compara el valor ingresado con el contenido de **mayor**. Si el número ingresado es superior, se actualiza el valor de **mayor**. Este proceso se repite hasta recibir los diez números.

Una vez terminadas las iteraciones, se muestra como salida el número mayor encontrado. Finalmente, se cierra el objeto **Scanner** para liberar el recurso utilizado para la entrada de datos.

3.6. Prueba realizada

Entrada:

Se ingresaron los siguientes diez números:

$-5, 12, 7, -20, 3, 15, 8, -2, 10, 6$

Al procesar los valores negativos como valores absolutos, los números considerados para la comparación fueron:

$5, 12, 7, 20, 3, 15, 8, 2, 10, 6$

Salida:

El programa mostró que el número mayor es:

20

3.7. Ejercicio 3: Generación de una tabla

Para resolver el ejercicio se creó la clase **Tabla** y se inicializó el método **main**. Debido a que los valores utilizados están definidos directamente en el programa, no fue necesario utilizar la clase **Scanner**.

Primero se mostró el encabezado de la tabla, formado por las columnas N , $10N$, $100N$ y $1000N$. Para separar cada columna se utilizó la tabulación mediante `\t`, permitiendo presentar los resultados de forma ordenada. El método `println` permite mostrar cada fila y realizar un salto de línea.

Posteriormente, se utilizó un ciclo `for` con una variable N inicializada en 1. El ciclo se ejecuta mientras N sea menor que 6, incrementando su valor en uno en cada iteración. En cada repetición se muestra el valor de N y los resultados de multiplicarlo por 10, 100 y 1000, utilizando la tabulación para separar las columnas.

Al finalizar las iteraciones, se obtiene una tabla con los valores correspondientes a $N = 1$ hasta $N = 5$.

3.8. Prueba realizada

Entrada:

Al no existir entrada por parte del usuario, los valores utilizados por el programa corresponden al recorrido establecido para N , desde 1 hasta 5.

Salida:

El programa muestra la siguiente tabla:

N	$10N$	$100N$	$1000N$
1	10	100	1000
2	20	200	2000
3	30	300	3000
4	40	400	4000
5	50	500	5000

4. Resultados

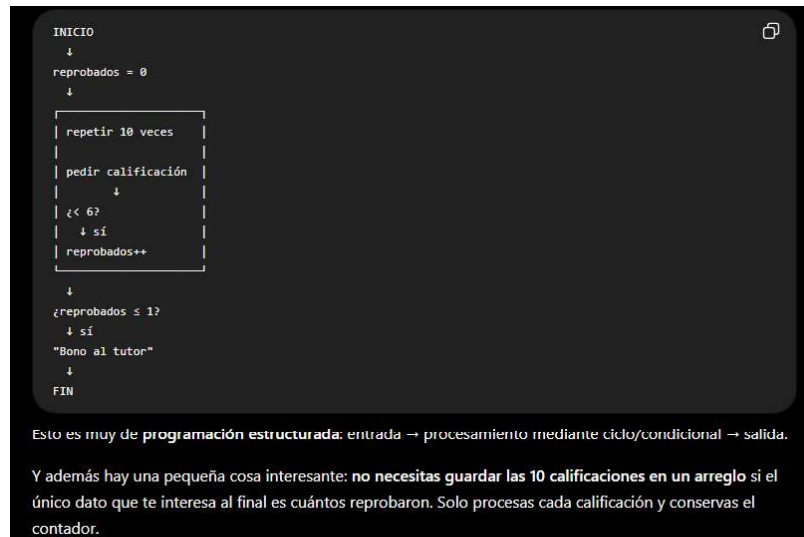


Figura 1: Al dar el prompt nos regresa un código similar al del profe y nos hace una comparación ya que se está entrando ya que le mencionamos que el paradigma a usar sería orientado a objetos.

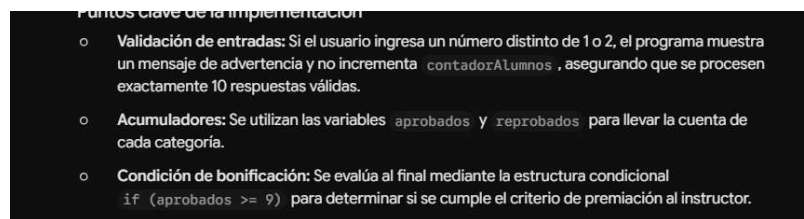


Figura 2: Nos cuenta que llevamos un contador de alumnos en los que se basen para saber cuántos aprobados y reprobados ahí y que reciba 10 alumnos.

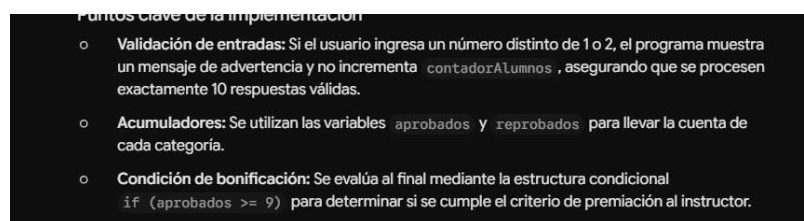


Figura 3: Diferenciamos que a uno le pone cada uno si está reprobado y aprobado y en la otra salida menciona que solo le importan los reprobados, no los demás.

En el siguiente enlace se encuentra disponible el archivo con el modelo gráfico de el ejercicio 2:

[Ver imagen Diagrama ejercicio 2.png en GitHub.](#)

Figura 4: En el modelo presentado se muestra el funcionamiento del ejercicio 2 para saber que numero es el mayor, sin aceptar numeros negativos, haciendo el ciclo hasta que se ingresen 10 numeros enteros positivos y te regresa el valor mayor con el mensaje $\therefore$ el numero mayor es num.

En el siguiente enlace se encuentra disponible el archivo con el modelo gráfico de el ejercicio 2: [Ver imagen Diagrama ejercicio 3.png en GitHub](#).

Figura 5: En el modelo se muestra el flujo del programa del ejercicio 3 donde se declaran variables y se realiza un ciclo para hacer tablas hasta el numero 5 y muestra como la tabla se llena multiplicando valores con n. Regresando una tabla

5. Conclusión

El análisis comparativo de las respuestas generadas por las LLM demuestra que la solución algorítmica depende directamente de qué tan específico es el prompt. Omitir información en el planteamiento obliga al modelo a asumir condiciones no especificadas. Por otra parte, en los programas de Java, se confirmó que la combinación de estructuras iterativas, while o for, y condicionales, if, permite procesar una serie numérica sin recurrir a arreglos. Aparte, el uso de secuencias de escape dentro del ciclo for demuestra su utilidad para organizar la salida en consola de forma tabular.

6. Referencias

[1] M. T. Goodrich, R. Tamassia y M. H. Goldwasser, Data Structures and Algorithms in Java, 6th ed. Hoboken, NJ, USA: Wiley, 2014.

[2] Oracle, “The Java® Language Specification, Chapter 14: Blocks, Statements, and Patterns,” Oracle. [En línea]. Disponible: Java Language Specification. [Accedido: 4-sep-2026].

[3] Oracle, “Class Scanner,” Java SE 26 API Documentation, Oracle. [En línea]. Disponible: Scanner Java API. [Accedido: 4-sep-2026].

[4] Oracle, “Class PrintStream,” Java SE 26 API Documentation, Oracle. [En línea]. Disponible en: PrintStream API Documentation. [Accedido: 4-sep-2026].